

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph dan Tree

Oleh:

Siti Nafisah

NIM. 2510817120009

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph dan Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Siti Nafisah
NIM : 2510817120009

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR	iii
DAFTAR TABEL.....	iv
Soal 1: Konversi Adjacency List ke Adjacency Matrix	1
A. Source Code	3
B. Output Program.....	5
C. Pembahasan.....	5
Soal 2: Konversi Adjacency Matrix ke Adjacency List	7
A. Source Code	9
B. Output Program.....	10
C. Pembahasan.....	10
Soal 3: BFS: Jarak Terpendek Keluar Labirin.....	13
A. Source Code	15
B. Output Program.....	17
C. Pembahasan.....	17
Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin.....	20
A. Source Code	22
B. Output Program.....	24
C. Pembahasan.....	24
Soal 5: Tree Traversal.....	26
A. Source Code	27
B. Output Program.....	31
C. Pembahasan.....	31
TAUTAN GIT.....	33

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Konversi Adjacency List ke Adjacency Matrix	5
Gambar 2. Screenshot Hasil Konversi Adjacency Matrix ke Adjacency List	10
Gambar 3. Screenshot Hasil BFS: Jarak Terpendek Keluar Labirin	17
Gambar 4. Screenshot Hasil DFS: Menghitung Semua Jalur Keluar Labirin.....	24
Gambar 5. Screenshot Hasil Tree Traversal	31

DAFTAR TABEL

Tabel 1 Source Code Soal1.cpp	3
Tabel 2 Source Code Soal2.cpp	9
Tabel 3 Source Code Soal3.cpp	15
Tabel 4 Source Code Soal4.cpp	22
Tabel 5 Source Code Soal5.cpp	27

Soal 1: Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi **adjacency matrix**. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
M
U1 V1 W1
U2 V2 W2
...
UM VM WM
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.

- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

```

      V1 V2 ... VN
V1 a11 a12 ... a1N
V2 a21 a22 ... a2N
...
VN aN1 aN2 ... aNN

```

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki **self-loop**.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6

B D 7	D 0 0 3 0 0
C B 1	E 0 0 0 0 0
C E 6	
D C 3	

A. Source Code

Tabel 1 Source Code Soal1.cpp

1	#include <iostream>
2	#include <vector>
3	using namespace std;
4	
5	int main() {
6	int N;
7	cin >> N;
8	
9	vector<char> vertex(N);
10	for (int i = 0; i < N; i++) {
11	cin >> vertex[i];
12	}
13	
14	vector<vector<int>> adjMatrix(N, vector<int>(N,
15	0));
16	
17	int M;
18	cin >> M;
19	
20	for (int i = 0; i < M; i++) {
	char U, V;

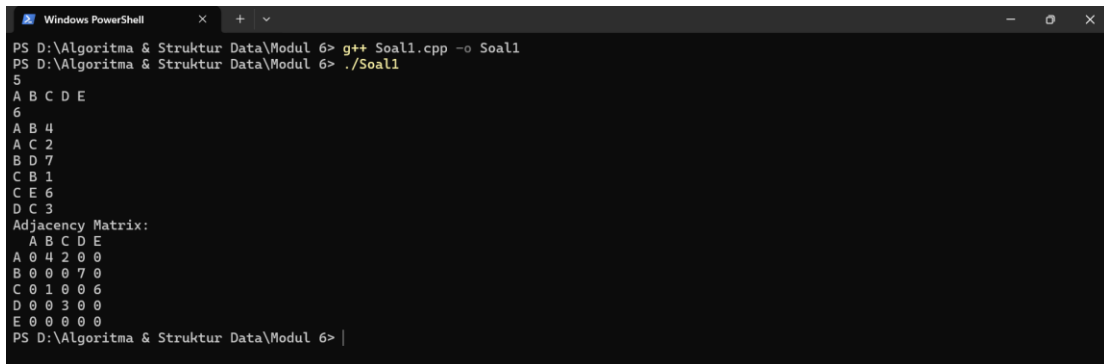

```

21         int W;
22         cin >> U >> V >> W;
23
24         int baris = -1;
25         int kolom = -1;
26
27         for (int j = 0; j < N; j++) {
28             if (vertex[j] == U) baris = j;
29             if (vertex[j] == V) kolom = j;
30         }
31
32         adjMatrix[baris][kolom] = W;
33     }
34
35     cout << "Adjacency Matrix:" << endl;
36     cout << " ";
37     for (int i = 0; i < N; i++) {
38         cout << " " << vertex[i];
39     }
40     cout << endl;
41
42     for (int i = 0; i < N; i++) {
43         cout << vertex[i];
44         for (int j = 0; j < N; j++) {
45             cout << " " << adjMatrix[i][j];
46         }
47         cout << endl;
48     }
49
50     return 0;

```

51	}
----	---

B. Output Program



```

PS D:\Algoritma & Struktur Data\Modul 6> g++ Soal1.cpp -o Soal1
PS D:\Algoritma & Struktur Data\Modul 6> ./Soal1
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS D:\Algoritma & Struktur Data\Modul 6> |

```

Gambar 1. Screenshot Hasil Konversi Adjacency List ke Adjacency Matrix

C. Pembahasan

Adjacency matrix merupakan salah satu cara merepresentasikan *graph* dalam bentuk matriks berukuran $N \times N$. Setiap baris menunjukkan *vertex* asal dan setiap kolom menunjukkan *vertex* tujuan. Jika terdapat *edge* dari suatu *vertex* ke *vertex* lain, maka sel yang sesuai akan berisi bobot *edge* tersebut. Sebaliknya, jika tidak terdapat hubungan antar *vertex*, nilainya bernilai 0. Representasi ini dipilih karena memudahkan penyimpanan hubungan antar *vertex* dalam bentuk tabel sehingga posisi setiap *edge* dapat diketahui secara langsung melalui indeks baris dan kolom.

Pada program ini, seluruh label *vertex* dibaca terlebih dahulu dan disimpan ke dalam `vector<char>`. Setelah itu dibuat matriks `adjMatrix` berukuran $N \times N$ yang diinisialisasi dengan nilai 0. Untuk setiap *edge* yang dibaca, program mencari posisi *vertex* asal dan *vertex* tujuan pada daftar *vertex* yang telah disimpan. Setelah posisi keduanya ditemukan, bobot *edge* disimpan ke dalam `adjMatrix[baris][kolom]` sehingga hubungan antar *vertex* dapat direpresentasikan dalam bentuk *adjacency matrix*.

Proses pencarian posisi *vertex* dilakukan dengan menelusuri daftar *vertex* yang telah disimpan sebelumnya. Pencarian tersebut dilakukan untuk setiap *edge* yang

dibaca. Dengan batasan jumlah *vertex* maksimum 10, proses tersebut masih bisa dilakukan dengan cepat dan efisien.

Dari sisi kompleksitas, *adjacency matrix* membutuhkan ruang sebesar $O(N^2)$ karena seluruh hubungan antar *vertex* disimpan dalam matriks berukuran $N \times N$. Sementara itu, untuk setiap *edge* program melakukan pencarian posisi *vertex* pada daftar *vertex* yang membutuhkan waktu $O(N)$. Karena proses tersebut dilakukan sebanyak M kali, kompleksitas waktu program adalah $O(M \times N)$. Karena jumlah *vertex* maksimum hanya 10, pencarian posisi *vertex* pada daftar *vertex* untuk setiap *edge* tidak memberikan biaya komputasi yang besar sehingga proses pembentukan *adjacency matrix* tetap berjalan dengan baik.

Pada contoh yang diberikan, *edge* A menuju B dengan bobot 4 menghasilkan nilai 4 pada baris A kolom B. Karena *graph* bersifat berarah, nilai pada baris B kolom A tetap 0 selama tidak terdapat *edge* dari B menuju A. Edge D menuju C dengan bobot 3 menghasilkan nilai 3 pada baris D kolom C. Selain itu, *vertex* E tidak memiliki *edge* keluar sehingga seluruh elemen pada baris E tetap bernilai 0. Hasil yang diperoleh menunjukkan bahwa setiap *edge* berhasil ditempatkan pada posisi baris dan kolom yang sesuai sehingga *adjacency matrix* yang dihasilkan telah merepresentasikan *graph* berarah berbobot dengan benar.

Soal 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s

Memory Limit: 64

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang direpresentasikan dalam bentuk **adjacency matrix** berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
A11 A12 ... A1N
A21 A22 ... A2N
...
AN1 AN2 ... ANN
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

V1 -> (X1,w1) , (X2,w2) , ...

V2 -> (X1,w1) , (X2,w2) , ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

- $2 \leq N \leq 1$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4) , (C,2) B -> (D,7) C -> (B,1) , (E,6) D -> (C,3) E -> -

A. Source Code

Tabel 2 Source Code Soal2.cpp

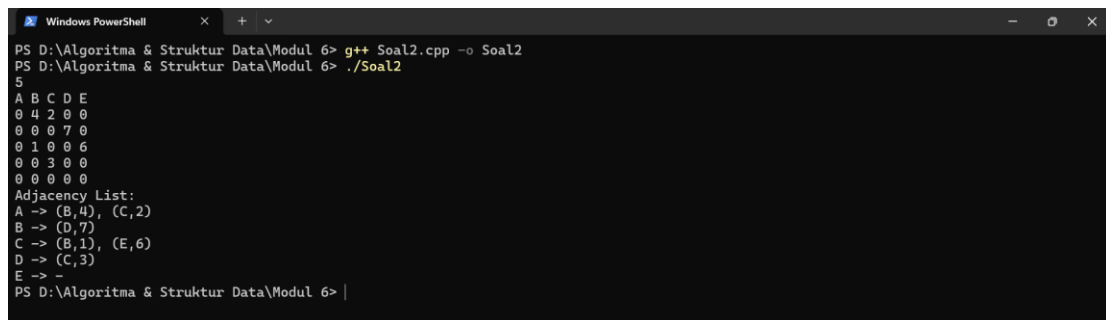
1	#include <iostream>
2	#include <vector>
3	using namespace std;
4	
5	int main() {
6	int N;
7	cin >> N;
8	
9	vector<char> vertex(N);
10	for (int i = 0; i < N; i++) {
11	cin >> vertex[i];
12	}
13	
14	vector<vector<int>> matrix(N, vector<int>(N));
15	for (int i = 0; i < N; i++) {
16	for (int j = 0; j < N; j++) {
17	cin >> matrix[i][j];
18	}
19	}
20	
21	cout << "Adjacency List:" << endl;
22	for (int i = 0; i < N; i++) {
23	cout << vertex[i] << " -> ";
24	bool adaEdge = false;
25	
26	for (int j = 0; j < N; j++) {
27	if (matrix[i][j] > 0) {
28	if (adaEdge)

```

29         cout << ", ";
30
31         cout << "(" << vertex[j] << "," <<
matrix[i][j] << ")";
32         adaEdge = true;
33     }
34 }
35
36     if (!adaEdge)
37         cout << "-";
38
39     cout << endl;
40 }
41
42     return 0;
43 }

```

B. Output Program



```

Windows PowerShell
PS D:\Algoritma & Struktur Data\Modul 6> g++ Soal2.cpp -o Soal2
PS D:\Algoritma & Struktur Data\Modul 6> ./Soal2
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS D:\Algoritma & Struktur Data\Modul 6> |

```

Gambar 2. Screenshot Hasil Konversi Adjacency Matrix ke Adjacency List

C. Pembahasan

Adjacency list merupakan salah satu cara merepresentasikan *graph* dengan menyimpan daftar *vertex* yang terhubung langsung dengan setiap *vertex*. Setiap *vertex* memiliki daftar tetangga beserta bobot *edge* yang menghubungkannya. Representasi

ini lebih mudah digunakan untuk melihat hubungan langsung antar *vertex* tanpa harus memeriksa seluruh elemen pada matriks

Pada program ini, *graph* diberikan dalam bentuk *adjacency matrix*. Setiap baris pada matriks menunjukkan *vertex* asal, sedangkan setiap kolom menunjukkan *vertex* tujuan. Program membaca seluruh nilai matriks, kemudian memeriksa setiap elemen pada setiap baris. Jika ditemukan nilai lebih dari 0, berarti terdapat *edge* dari *vertex* pada baris tersebut menuju *vertex* pada kolom yang bersesuaian. Vertex tujuan dan bobot *edge* kemudian ditampilkan dalam format *adjacency list*. Sebaliknya, jika seluruh nilai pada suatu baris bernilai 0, maka *vertex* tersebut tidak memiliki *edge* keluar dan program mencetak tanda "-".

Karena seluruh elemen pada matriks harus diperiksa, proses konversi dilakukan dengan menelusuri setiap baris dan kolom yang ada pada matriks. Karena seluruh elemen pada matriks harus diperiksa satu per satu, program melakukan penelusuran terhadap seluruh pasangan baris dan kolom. Pada batasan soal yang diberikan, pemeriksaan seluruh elemen matriks masih dapat dilakukan secara langsung tanpa menimbulkan biaya komputasi yang besar.

Dari sisi kompleksitas, program harus memeriksa seluruh elemen pada matriks berukuran $N \times N$. Oleh karena itu, kompleksitas waktu yang dibutuhkan adalah $O(N^2)$. Sementara itu, kebutuhan ruang utama berasal dari penyimpanan *adjacency matrix* yang juga berukuran $N \times N$, sehingga kompleksitas ruangnya adalah $O(N^2)$. Dengan ukuran input yang kecil sesuai batasan soal, penggunaan waktu dan memori tersebut masih cukup efisien.

Pada contoh yang diberikan, baris *vertex* A memiliki nilai positif pada kolom B dan C, yaitu 4 dan 2. Oleh karena itu, *adjacency list* untuk *vertex* A adalah (B, 4) dan (C, 2). *Vertex* C juga memiliki hubungan menuju *vertex* B dan E sehingga ditampilkan sebagai (B, 1) dan (E, 6). Sementara itu, seluruh elemen pada baris *vertex* E bernilai 0 sehingga *vertex* E tidak memiliki *edge* keluar dan hasil yang ditampilkan adalah E -> -. Hasil tersebut menunjukkan bahwa setiap hubungan pada *adjacency matrix* berhasil dikonversi ke bentuk *adjacency list* sesuai dengan struktur *graph* yang diberikan.

Soal 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s

Memory Limit: 64

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.

- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0000100000 1110101110 0010000010 0111111010 0000001000 1111101110 0000100010 0110001000 00 79	16

A. Source Code

Tabel 3 Source Code Soal3.cpp

1	#include <iostream>
2	#include <vector>
3	#include <queue>
4	using namespace std;
5	
6	int main() {
7	int R, C;
8	cin >> R >> C;
9	
10	vector<vector<int>> grid(R, vector<int>(C));
11	for (int i = 0; i < R; i++) {
12	for (int j = 0; j < C; j++) {
13	cin >> grid[i][j];

```

14         }
15     }
16
17     int SR, SC;
18     int FR, FC;
19
20     cin >> SR >> SC;
21     cin >> FR >> FC;
22
23     vector<vector<bool>> visited(R, vector<bool>(C,
24 false));
25     queue<pair<pair<int, int>, int>> q;
26
27     q.push({{SR, SC}, 0});
28     visited[SR][SC] = true;
29
30     int baris[] = {-1, 1, 0, 0};
31     int kolom[] = {0, 0, -1, 1};
32
33     while (!q.empty()) {
34         int r = q.front().first.first;
35         int c = q.front().first.second;
36         int jarak = q.front().second;
37         q.pop();
38
39         if (r == FR && c == FC) {
40             cout << jarak << endl;
41             return 0;
42         }

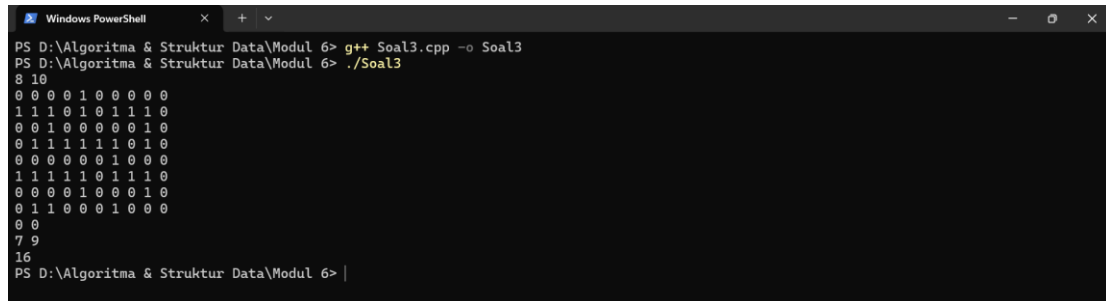
```

```

43         for (int i = 0; i < 4; i++) {
44             int nr = r + baris[i];
45             int nc = c + kolom[i];
46
47             if (nr >= 0 && nr < R &&
48                 nc >= 0 && nc < C &&
49                 grid[nr][nc] == 0 &&
50                 !visited[nr][nc]) {
51
52                 visited[nr][nc] = true;
53                 q.push({nr, nc}, jarak + 1);
54             }
55         }
56     }
57
58     cout << -1 << endl;
59     return 0;
60 }

```

B. Output Program



```

PS D:\Algoritma & Struktur Data\Modul 6> g++ Soal3.cpp -o Soal3
PS D:\Algoritma & Struktur Data\Modul 6> ./Soal3
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
PS D:\Algoritma & Struktur Data\Modul 6>

```

Gambar 3. Screenshot Hasil BFS: Jarak Terpendek Keluar Labirin

C. Pembahasan

Breadth-First Search (BFS) merupakan algoritma penelusuran yang bekerja dengan mengunjungi simpul berdasarkan urutan jarak terdekat terlebih dahulu. Pada

permasalahan pencarian jalur terpendek, BFS sering digunakan karena mampu menemukan jarak minimum pada *graph* atau *grid* yang memiliki bobot seragam. Proses penelusuran dilakukan secara bertahap dengan memeriksa seluruh kemungkinan posisi yang dapat dicapai pada jarak tertentu sebelum berpindah ke jarak berikutnya.

Pada program ini, labirin direpresentasikan dalam bentuk *grid* yang terdiri dari sel bernilai 0 dan 1. Nilai 0 menunjukkan jalan yang dapat dilewati, sedangkan nilai 1 menunjukkan dinding. BFS diterapkan dengan menggunakan *queue* untuk menyimpan posisi yang akan diproses. Posisi awal dimasukkan terlebih dahulu ke dalam *queue*, kemudian program memeriksa empat kemungkinan arah pergerakan, yaitu atas, bawah, kiri, dan kanan. Setiap sel yang valid dan belum pernah dikunjungi akan dimasukkan ke dalam *queue* dengan jarak yang bertambah satu langkah. Proses ini terus dilakukan hingga posisi tujuan ditemukan atau tidak ada lagi posisi yang dapat dieksplorasi.

Pada BFS, setiap sel yang dapat dilewati akan dikunjungi paling banyak satu kali karena program menggunakan penanda *visited* untuk mencegah kunjungan berulang. Oleh karena itu, jumlah proses yang dilakukan bergantung pada banyaknya sel dalam *grid*. Dengan Batasan ukuran *grid* yang diberikan pada soal, BFS dapat menemukan jalur terpendek dengan efisien tanpa perlu memeriksa posisi yang sama berulang kali.

Dari sisi kompleksitas, setiap sel pada *grid* akan dikunjungi paling banyak satu kali karena program menggunakan array *visited* untuk mencegah kunjungan berulang. Selain itu, dari setiap sel program hanya memeriksa empat kemungkinan arah pergerakan. Oleh karena itu, kompleksitas waktu BFS pada program ini adalah $O(R \times C)$, dengan R menyatakan jumlah baris dan C menyatakan jumlah kolom pada *grid*. Sementara itu, kompleksitas ruang juga sebesar $O(R \times C)$ karena program menyimpan data *grid*, array *visited*, serta *queue* yang pada kondisi tertentu dapat menampung sejumlah sel yang masih menunggu untuk diproses.

Pada contoh yang diberikan, posisi awal berada pada koordinat $(0, 0)$ dan posisi tujuan berada pada koordinat $(7, 9)$. Labirin memiliki beberapa percabangan

dan jalan buntu sehingga tidak semua jalur akan mengarah ke tujuan. Dengan BFS, setiap posisi diproses berdasarkan jarak terdekat dari titik awal sehingga jalur yang ditemukan pertama kali menuju tujuan merupakan jalur dengan jumlah langkah minimum. Hasil yang diperoleh adalah 16 langkah, yang menunjukkan bahwa BFS berhasil menemukan jarak terpendek dari posisi awal menuju posisi tujuan sesuai dengan kondisi labirin yang diberikan.

Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 1.0 s

Memory Limit: 64

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif. Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C
G21 G22 ... G2C
...
GR1 GR2 ... GRC
SR SC
FR FC
```

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.

- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0	4

1 1 0 0 0 0	
0 0	
4 5	

A. Source Code

Tabel 4 Source Code Soal4.cpp

1	#include <iostream>
2	#include <vector>
3	using namespace std;
4	
5	int R, C;
6	vector<vector<int>> grid;
7	vector<vector<bool>> visited;
8	
9	int FR, FC;
10	int totalJalur = 0;
11	
12	int baris[] = {-1, 1, 0, 0};
13	int kolom[] = {0, 0, -1, 1};
14	
15	void dfs(int r, int c) {
16	if (r == FR && c == FC) {
17	totalJalur++;
18	return;
19	}
20	
21	visited[r][c] = true;
22	
23	for (int i = 0; i < 4; i++) {
24	int barisBaru = r + baris[i];

```

25         int kolomBaru = c + kolom[i];
26
27         if (barisBaru >= 0 && barisBaru < R &&
28             kolomBaru >= 0 && kolomBaru < C &&
29             grid[barisBaru][kolomBaru] == 0 &&
30             !visited[barisBaru][kolomBaru]) {
31
32             dfs(barisBaru, kolomBaru);
33         }
34     }
35
36     visited[r][c] = false;
37 }
38
39 int main() {
40     cin >> R >> C;
41
42     grid.assign(R, vector<int>(C));
43     visited.assign(R, vector<bool>(C, false));
44
45     for (int i = 0; i < R; i++) {
46         for (int j = 0; j < C; j++) {
47             cin >> grid[i][j];
48         }
49     }
50
51     int SR, SC;
52
53     cin >> SR >> SC;
54     cin >> FR >> FC;

```

55	
56	dfs(SR, SC);
57	
58	cout << totalJalur << endl;
59	
60	return 0;
61	}

B. Output Program

```

Windows PowerShell
PS D:\Algoritma & Struktur Data\Modul 6> g++ Soal4.cpp -o Soal4
PS D:\Algoritma & Struktur Data\Modul 6> ./Soal4
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS D:\Algoritma & Struktur Data\Modul 6>

```

Gambar 4. Screenshot Hasil DFS: Menghitung Semua Jalur Keluar Labirin

C. Pembahasan

Depth-First Search (DFS) merupakan algoritma penelusuran yang bekerja dengan menjelajahi suatu jalur sedalam mungkin sebelum kembali ke percabangan sebelumnya. Pada permasalahan ini, DFS digunakan untuk menghitung seluruh jalur sederhana dari posisi awal menuju posisi tujuan. Agar satu jalur tidak mengunjungi sel yang sama lebih dari satu kali, digunakan teknik *backtracking* yang memungkinkan program kembali ke kondisi sebelumnya setelah suatu jalur selesai diperiksa.

Pada program ini, labirin direpresentasikan dalam bentuk *grid* yang terdiri dari sel bernilai 0 dan 1. Nilai 0 menunjukkan jalan yang dapat dilalui, sedangkan nilai 1 menunjukkan dinding. DFS dimulai dari posisi awal dan mencoba bergerak ke empat arah, yaitu atas, bawah, kiri, dan kanan. Setiap sel yang dikunjungi akan ditandai agar tidak dilewati kembali dalam jalur yang sama. Jika posisi tujuan berhasil dicapai, jumlah jalur valid akan ditambah satu. Setelah seluruh kemungkinan dari suatu posisi

selesai diperiksa, tanda kunjungan dihapus kembali sehingga sel tersebut dapat digunakan pada percabangan jalur lainnya. Proses inilah yang disebut *backtracking*.

Pada DFS, setiap kemungkinan jalur akan dieksplorasi hingga mencapai tujuan atau tidak dapat dilanjutkan lagi. Karena program harus mencoba seluruh jalur yang mungkin, jumlah proses yang dilakukan bergantung pada banyaknya percabangan yang ada pada labirin. Dengan Batasan ukuran *grid* maksimum 7×7 serta jumlah jalur valid yang dibatasi pada soal, proses pencarian masih dapat dilakukan dengan baik tanpa melebihi batas yang diberikan.

Dari sisi kompleksitas, program menggunakan DFS dengan teknik *backtracking* untuk mengeksplorasi seluruh kemungkinan jalur dari posisi awal menuju posisi tujuan. Berbeda dengan DFS biasa yang hanya menelusuri setiap simpul satu kali, pada program ini suatu sel dapat dikunjungi kembali melalui jalur yang berbeda setelah proses *backtracking* dilakukan. Akibatnya, jumlah proses yang dilakukan bergantung pada banyaknya percabangan dan kemungkinan jalur yang terdapat pada labirin. Sementara itu, kebutuhan ruang utama berasal dari penyimpanan *grid*, *array visited*, serta pemanggilan fungsi rekursif selama proses DFS berlangsung. Dengan batasan ukuran *grid* maksimum 7×7 dan jumlah jalur valid yang dibatasi pada soal, penggunaan waktu dan memori masih dapat memenuhi batas yang diberikan.

Pada contoh yang diberikan, posisi awal berada pada koordinat $(0, 0)$ dan posisi tujuan berada pada koordinat $(4, 5)$. Labirin memiliki lebih dari satu jalur yang dapat mencapai tujuan serta beberapa cabang yang tidak menghasilkan solusi, seperti jalur yang mengarah ke sel $(0, 5)$. Dengan DFS dan *backtracking*, seluruh kemungkinan jalur sederhana diperiksa satu per satu hingga tidak ada lagi jalur yang dapat dieksplorasi. Hasil yang diperoleh adalah 4 jalur valid dari posisi awal menuju posisi tujuan. Hal ini menunjukkan bahwa DFS berhasil menghitung seluruh jalur yang memenuhi syarat tanpa menghitung jalur yang mengunjungi sel yang sama lebih dari satu kali.

Soal 5: Tree Traversal

Time Limit: 1.0 s

Memory Limit: 64

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **Red-Black Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTtree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

$\dots$

T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTtree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.

- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

```
[Preorder] : daftar_angka
[Inorder]   : daftar_angka
[Postorder] : daftar_angka
```

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-1000000000 \leq A_i \leq 1000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder]: 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder]: 20 40 30 60 80 70 50
ALL	

A. Source Code

Tabel 5 Source Code Soal5.cpp

1	#include <iostream>
---	---------------------


```

2  #include <string>
3  #include "RedBlackTree.h"
4  using namespace std;
5
6  void preorder(const RedBlackTree::Node* node,
7               const RedBlackTree::Node* nil)
8  {
9      if (node == nil) return;
10
11     cout << node->key << " ";
12     preorder(node->left, nil);
13     preorder(node->right, nil);
14 }
15
16 void inorder(const RedBlackTree::Node* node,
17              const RedBlackTree::Node* nil)
18 {
19     if (node == nil) return;
20
21     inorder(node->left, nil);
22     cout << node->key << " ";
23     inorder(node->right, nil);
24 }
25
26 void postorder(const RedBlackTree::Node* node,
27                const RedBlackTree::Node* nil)
28 {
29     if (node == nil) return;
30
31     postorder(node->left, nil);

```

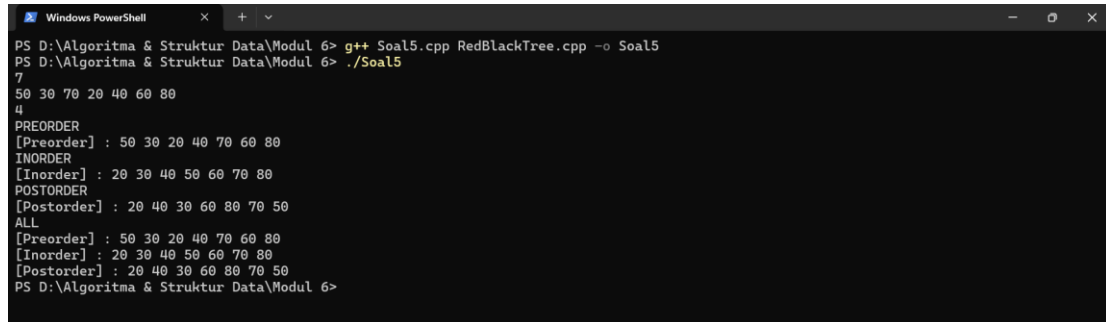
```

32     postorder(node->right, nil);
33     cout << node->key << " ";
34 }
35
36 int main() {
37     int N;
38     cin >> N;
39
40     RedBlackTree tree;
41
42     for (int i = 0; i < N; i++) {
43         int x;
44         cin >> x;
45         if (!tree.contains(x))
46             tree.insert(x);
47     }
48
49     int Q;
50     cin >> Q;
51
52     if (tree.empty()) {
53         cout << "Tree kosong. Tidak ada yang bisa
ditampilkan." << endl;
54         return 0;
55     }
56
57     while (Q--) {
58         string traversal;
59         cin >> traversal;
60

```

61	if (traversal == "PREORDER") {
62	cout << "[Preorder] : ";
63	preorder(tree.root(), tree.nil());
64	cout << endl;
65	} else if (traversal == "INORDER") {
66	cout << "[Inorder] : ";
67	inorder(tree.root(), tree.nil());
68	cout << endl;
69	} else if (traversal == "POSTORDER") {
70	cout << "[Postorder] : ";
71	postorder(tree.root(), tree.nil());
72	cout << endl;
73	} else if (traversal == "ALL") {
74	cout << "[Preorder] : ";
75	preorder(tree.root(), tree.nil());
76	cout << endl;
77	
78	cout << "[Inorder] : ";
79	inorder(tree.root(), tree.nil());
80	cout << endl;
81	
82	cout << "[Postorder] : ";
83	postorder(tree.root(), tree.nil());
84	cout << endl;
85	}
86	}
87	
88	return 0;
89	}

B. Output Program



```
PS D:\Algoritma & Struktur Data\Modul 6> g++ Soal5.cpp RedBlackTree.cpp -o Soal5
PS D:\Algoritma & Struktur Data\Modul 6> ./Soal5
7
50 30 70 20 40 60 80
4
PREORDER
[Preorder] : 50 30 20 40 70 60 80
INORDER
[Inorder] : 20 30 40 50 60 70 80
POSTORDER
[Postorder] : 20 40 30 60 80 70 50
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
PS D:\Algoritma & Struktur Data\Modul 6>
```

Gambar 5. Screenshot Hasil Tree Traversal

C. Pembahasan

Red-Black Tree merupakan salah satu jenis *binary search tree* yang memiliki aturan tambahan mengenai warna *node* untuk menjaga keseimbangan tree. Dengan kondisi tree yang tetap seimbang, proses pencarian, penyisipan, dan traversal dapat dilakukan dengan lebih efisien. Pada soal ini, data dimasukkan ke dalam *Red-Black Tree* secara berurutan, kemudian dilakukan traversal sesuai dengan query yang diberikan.

Program terlebih dahulu membaca seluruh bilangan yang akan dimasukkan ke dalam tree. Sebelum melakukan penyisipan, program memeriksa apakah nilai tersebut sudah ada di dalam tree. Jika ditemukan nilai yang sama, data tersebut tidak dimasukkan kembali sehingga tidak terjadi duplikasi *node*. Setelah tree terbentuk, program membaca query traversal yang diminta. Traversal yang tersedia terdiri dari *preorder*, *inorder*, dan *postorder*. *Preorder* mengunjungi *root* terlebih dahulu kemudian *subtree* kiri dan *subtree* kanan. *Inorder* mengunjungi *subtree* kiri, kemudian *root*, lalu *subtree* kanan. *Postorder* mengunjungi *subtree* kiri, *subtree* kanan, kemudian *root*.

Pada proses traversal, setiap *node* pada tree yang dikunjungi tepat satu kali sesuai urutan traversal yang dipilih. Oleh karena itu, banyaknya proses yang dilakukan bergantung pada jumlah *node* yang terdapat pada tree. Dengan jumlah data maksimum yang diberikan pada soal, proses traversal masih dapat dilakukan dengan efisien untuk seluruh query yang diminta.

Dari sisi kompleksitas, setiap proses traversal mengunjungi seluruh *node* pada *tree* tepat satu kali sehingga kompleksitas waktunya adalah $O(N)$, dengan N menyatakan jumlah *node* pada *tree*. Sementara itu, traversal dilakukan secara rekursif sehingga membutuhkan ruang tambahan pada *call stack*. Karena *Red-Black Tree* menjaga tinggi *tree* tetap seimbang, kompleksitas ruang traversal adalah $O(\log N)$. Dengan jumlah data yang diberikan pada soal, proses traversal dapat dilakukan dengan efisien untuk seluruh query yang diminta.

Pada contoh yang diberikan, bilangan dimasukkan secara berurutan sehingga terbentuk struktur *Red-Black Tree* yang sesuai dengan aturan *binary search tree*. Nilai 50 menjadi *root*, sedangkan nilai yang lebih kecil ditempatkan pada *subtree* kiri dan nilai yang lebih besar ditempatkan pada *subtree* kanan. Hasil traversal preorder menunjukkan urutan kunjungan *root*, *subtree* kiri, dan *subtree* kanan. Hasil traversal postorder menunjukkan urutan kunjungan *subtree* kiri, *subtree* kanan, kemudian *root*. Sementara itu, traversal inorder menghasilkan urutan 20 30 40 50 60 70 80. Hasil tersebut menunjukkan bahwa seluruh data tersusun sesuai aturan *binary search tree* karena traversal inorder selalu menghasilkan urutan data dari nilai terkecil hingga terbesar.

Dengan menggunakan traversal pada *Red-Black Tree*, struktur data yang telah dibentuk dapat ditampilkan dari beberapa sudut pandang yang berbeda sesuai kebutuhan. Hasil yang diperoleh pada setiap query menunjukkan bahwa proses traversal telah berjalan sesuai dengan definisi preorder, inorder, dan postorder.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-snafisahh>